WEEK 6 – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM.

-S.Kaviya E25AI119

Scenario:

The Art Work Shop needs to maintain customer reviews and ratings for its products. The Review table stores customer feedback, product ratings, and review dates. This module helps the shop identify highly rated products, analyze customer satisfaction, and find products that require improvement.

Creating table:

```
SQL> CREATE TABLE Review (
  2       Review_ID VARCHAR2(10),
  3       Customer_ID VARCHAR2(20),
  4       Product_ID VARCHAR2(7),
  5       Rating NUMBER(1),
  6       Review_Text VARCHAR2(500),
  7       Review_Date DATE
  8  );

Table created.
```

Applying Constraints:

```
SQL> ALTER TABLE Review ADD CONSTRAINT PK_REVIEW PRIMARY KEY (Review_ID);
Table altered.
SQL> ALTER TABLE Review ADD CONSTRAINT FK_REVIEW_CUSTOMER FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID);
Table altered.
SQL> ALTER TABLE Review ADD CONSTRAINT FK_REVIEW_PRODUCT FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID);
Table altered.
SQL> ALTER TABLE Review ADD CONSTRAINT CHK_REVIEW_RATING CHECK (Rating BETWEEN 1 AND 5);
Table altered.
SQL> ALTER TABLE Review MODIFY (Customer_ID NOT NULL);
Table altered.
SQL> ALTER TABLE Review MODIFY (Product_ID NOT NULL);
Table altered.
SQL> ALTER TABLE Review MODIFY (Rating NOT NULL);
Table altered.
```

Primary Key: Review_ID uniquely identifies each review.

Foreign Key: Customer_ID references the Customer table.

Foreign Key: Product_ID references the Product table.

NOT NULL Constraint: Customer_ID, Product_ID, and Rating cannot contain NULL values.

CHECK Constraint: Rating must be between 1 and 5.

Establishing Relationships:

- Review.Customer_ID → Customer.Customer_ID
- Review.Product_ID → Product.Product_ID
- Applied foreign key constraints to maintain referential integrity.

Inserting Values:

1.Store Customer Feedback and Ratings

- Add reviews for purchased products

- Store customer ratings from 1 to 5

```
SQL> INSERT INTO Review VALUES ('REV0001', 'C-1001', 'P1001', 4, 'Good quality', DATE '2026-09-15');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0002', 'C-1002', 'P1002', 5, 'Excellent product', DATE '2026-09-18');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0003', 'C-1003', 'P1003', 4, 'Good quality product', DATE '2026-09-19');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0004', 'C-1004', 'P1004', 3, 'Average product', DATE '2026-09-20');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0005', 'C-1005', 'P1005', 5, 'Really useful product', DATE '2026-09-21');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0006', 'C-1001', 'P1006', 4, 'Worth the price', DATE '2026-09-22');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0007', 'C-1002', 'P1003', 2, 'Quality can be better', DATE '2026-09-23');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0008', 'C-1003', 'P1001', 5, 'Very satisfied', DATE '2026-09-24');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0009', 'C-1004', 'P1002', 3, 'Decent product', DATE '2026-09-25');
1 row created.
SQL> INSERT INTO Review VALUES ('REV0010', 'C-1005', 'P1004', 4, 'Good performance', DATE '2026-09-26');
1 row created.
```

```
SQL> SELECT * FROM Review;

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0001    C-1001               P1001            4
Good quality
15-SEP-26

REV0002    C-1002               P1002            5
Excellent product
18-SEP-26

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV0003    C-1003               P1003            4
Good quality product
19-SEP-26

REV0004    C-1004               P1004            3
Average product

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
20-SEP-26

REV0005    C-1005               P1005            5
Really useful product
21-SEP-26
```

```
REV0006    C-1001               P1006            4

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Worth the price
22-SEP-26

REV0007    C-1002               P1003            2
Quality can be better
23-SEP-26


REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0008    C-1003               P1001            5
Very satisfied
24-SEP-26

REV0009    C-1004               P1002            3
Decent product
25-SEP-26

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV0010    C-1005               P1004            4
Good performance
26-SEP-26
```

- Update review commands if required

```
SQL> UPDATE Review SET Review_Text = 'Very good product' WHERE Review_ID = 'REV0003';

1 row updated.
```

- Remove inappropriate or invalid reviews

```
SQL> DELETE FROM Review WHERE Review_ID = 'REV0007';

1 row deleted.
```

2.Retrieve product review details

- Display all reviews for a product

```
SQL> SELECT * FROM Review WHERE Product_ID = 'P1001';

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0001    C-1001               P1001            4
Good quality
15-SEP-26

REV0008    C-1003               P1001            5
Very satisfied
24-SEP-26
```

- Display customer name With their reviews

```
SQL> SELECT Review.Review_ID, Customer.First_Name, Customer.Last_Name, Review.Product_ID, Review.Rating, Review.Review_T
ext, Review.Review_Date FROM Review JOIN Customer ON Review.Customer_ID = Customer.Customer_ID;

REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0001    Beema               ponting             P1001            4
Good quality
15-SEP-26

REV0002    priya               sakthi              P1002            5
Excellent product
18-SEP-26

REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV0003    Arun                kumar               P1003            4
Very good product
19-SEP-26
```

```
REV0004    Anu                 Priya               P1004            3
Average product

REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
20-SEP-26

REV0005    Reena               das                 P1005            5
Really useful product
21-SEP-26

REV0006    Beema               ponting             P1006            4

REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Worth the price
22-SEP-26

REV0008    Arun                kumar               P1001            5
Very satisfied
24-SEP-26


REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0009    Anu                 Priya               P1002            3
Decent product
25-SEP-26
```

```
---------
REV0009    Anu                 Priya               P1002           3
Decent product
25-SEP-26

REV0010    Reena               das                 P1004           4
Good performance
26-SEP-26

REVIEW_ID  FIRST_NAME          LAST_NAME           PRODUCT     RATING
---------- ------------------- ------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------


9 rows selected.
```

- Find product having maximum reviews

```
SQL> SELECT Review.Product_ID, Product.Product_Name, COUNT(Review.Review_ID) AS Total_Reviews FROM Review JOIN Product O
N Review.Product_ID = Product.Product_ID GROUP BY Review.Product_ID, Product.Product_Name ORDER BY Total_Reviews DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------
P1001
Laptops
           2

P1002
Croptops and shirts
           2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------

P1004
Television
           2
```

- Display recent customer feedback

```
             2

SQL> SELECT * FROM (SELECT * FROM Review ORDER BY Review_Date DESC) WHERE ROWNUM <= 5;

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0010    C-1005               P1004            4
Good performance
26-SEP-26

REV0009    C-1004               P1002            3
Decent product
25-SEP-26

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV0008    C-1003               P1001            5
Very satisfied
24-SEP-26

REV0006    C-1001               P1006            4
Worth the price

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
22-SEP-26

REV0005    C-1005               P1005            5
Really useful product
```

- Retrieve reviews with rating above 4

```
SQL> SELECT * FROM Review WHERE Rating > 4;

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV0002    C-1002               P1002            5
Excellent product
18-SEP-26

REV0005    C-1005               P1005            5
Really useful product
21-SEP-26

REVIEW_ID  CUSTOMER_ID          PRODUCT     RATING
---------- -------------------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV0008    C-1003               P1001            5
Very satisfied
24-SEP-26
```

3.Calculate average product ratings

Using aggregate functions to Analyse ratings

- Calculate average ratings for each product

```
SQL> SELECT Product.Product_ID, Product.Product_Name, AVG(Review.Rating) AS Average_Rating FROM Product JOIN Review ON P
roduct.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
P1001
Laptops
           4.5

P1005
Unruled notes
             5

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------

P1006
Tiles
             4

P1002
```

```
P1002
Croptops and shirts

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
             4

P1003
Fish tank
             4

P1004

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
Television
           3.5


6 rows selected.
```

- Count the number of Reviews per products

```
SQL> SELECT Product.Product_ID, Product.Product_Name, COUNT(Review.Review_ID) AS Total_Reviews FROM Product JOIN Review
ON Product.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------
P1001
Laptops
             2

P1005
Unruled notes
             1

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------

P1006
Tiles
             1

P1002
```

```
P1002
Croptops and shirts

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------
            2

P1003
Fish tank
            1

P1004

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_REVIEWS
-------------
Television
            2


6 rows selected.
```

- Find highest rated products

```
SQL> SELECT Product.Product_ID, Product.Product_Name, AVG(Review.Rating) AS Average_Rating FROM Product JOIN Review ON P
roduct.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name HAVING AVG(Review.Rating) = (SEL
ECT MAX(AVG(Rating)) FROM Review GROUP BY Product_ID);

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
P1005
Unruled notes
           5
```

- Identify products with average ratings above 4

```
SQL> SELECT Product.Product_ID, Product.Product_Name, AVG(Review.Rating) AS Average_Rating FROM Product JOIN Review ON P
roduct.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name HAVING AVG(Review.Rating) > 4;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
P1001
Laptops
           4.5

P1005
Unruled notes
             5
```

4.Identity highly related products

Generate product rating reports

Report 1 : product rating analysis

Display:

- Product name
- Number of Reviews
- Average rating

```
SQL> SELECT Product.Product_Name, COUNT(Review.Review_ID) AS Number_Of_Reviews, ROUND(AVG(Review.Rating), 2) AS Average_Rating FROM Product LEFT JOIN Review
 ON Product.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name ORDER BY Average_Rating DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Unruled notes
                1              5

Laptops
                2            4.5

Tiles
                1              4


PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Croptops and shirts
                2              4

Fish tank
                1              4

Television
                2            3.5


6 rows selected.
```

Report 2: Customer feedback analysis

Find :

- Most reviewed products

```
SQL> SELECT Product.Product_Name, COUNT(Review.Review_ID) AS Number_Of_Reviews FROM Product LEFT JOIN Review ON Product.Product_ID = Review.Product_ID GROUP
 BY Product.Product_ID, Product.Product_Name ORDER BY Number_Of_Reviews DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS
-----------------
Laptops
                2

Croptops and shirts
                2

Television
                2


PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS
-----------------
Fish tank
                1

Unruled notes
                1

Tiles
                1


6 rows selected.
```

- Highly rated products

```
SQL> SELECT Product.Product_Name, ROUND(AVG(Review.Rating), 2) AS Average_Rating FROM Product JOIN Review ON Product.Pro
duct_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name HAVING AVG(Review.Rating) > 4 ORDER BY Ave
rage_Rating DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
Unruled notes
              5

Laptops
            4.5
```

- Products requiring Improvement

```
SQL> SELECT Product.Product_Name, COUNT(Review.Review_ID) AS Review_Count, ROUND(AVG(Review.Rating), 2) AS Average_Ratin
g FROM Product JOIN Review ON Product.Product_ID = Review.Product_ID GROUP BY Product.Product_ID, Product.Product_Name H
AVING AVG(Review.Rating) < 3 ORDER BY Average_Rating;

no rows selected
```

Report 3 : Rating distribution

- Number of 5 star rating

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings FROM Review WHERE Rating = 5;

FIVE_STAR_RATINGS
-----------------
                3
```

- Number of 4 star rating

```
SQL> SELECT COUNT(*) AS Four_Star_Ratings FROM Review WHERE Rating = 4;

FOUR_STAR_RATINGS
-----------------
                4
```

- Number of low- rated products

```
SQL> SELECT COUNT(*) AS Low_Rated_Products FROM (SELECT Product_ID FROM Review GROUP BY Product_ID HAVING AVG(Rating) < 3);

LOW_RATED_PRODUCTS
------------------
                 0
```

- Complete rating

```
SQL> SELECT Rating, COUNT(*) AS Number_Of_Ratings FROM Review GROUP BY Rating ORDER BY Rating DESC;

    RATING NUMBER_OF_RATINGS
---------- -----------------
         5                 3
         4                 4
         3                 2
```